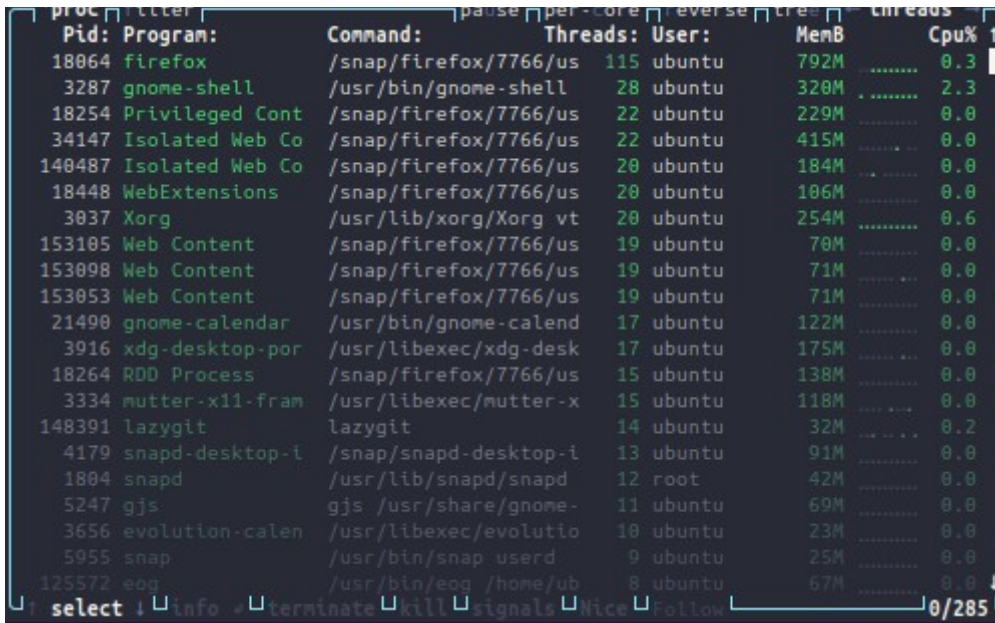


Simple Measurements – Idleness

1.

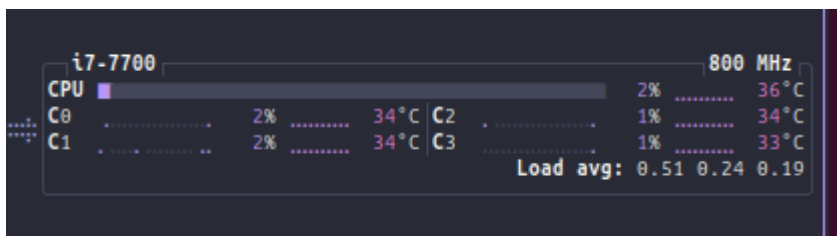


Pid	Program	Command	Threads	User	MemB	Cpu%
18064	firefox	/snap/firefox/7766/us	115	ubuntu	792M	0.3
3287	gnome-shell	/usr/bin/gnome-shell	28	ubuntu	320M	2.3
18254	Privileged Cont	/snap/firefox/7766/us	22	ubuntu	229M	0.0
34147	Isolated Web Co	/snap/firefox/7766/us	22	ubuntu	415M	0.0
140487	Isolated Web Co	/snap/firefox/7766/us	20	ubuntu	184M	0.0
18448	WebExtensions	/snap/firefox/7766/us	20	ubuntu	106M	0.0
3037	Xorg	/usr/lib/xorg/Xorg vt	20	ubuntu	254M	0.6
153105	Web Content	/snap/firefox/7766/us	19	ubuntu	70M	0.0
153098	Web Content	/snap/firefox/7766/us	19	ubuntu	71M	0.0
153053	Web Content	/snap/firefox/7766/us	19	ubuntu	71M	0.0
21490	gnome-calendar	/usr/bin/gnome-calend	17	ubuntu	122M	0.0
3916	xdg-desktop-por	/usr/libexec/xdg-desk	17	ubuntu	175M	0.0
18264	RDD Process	/snap/firefox/7766/us	15	ubuntu	138M	0.0
3334	mutter-x11-fram	/usr/libexec/mutter-x	15	ubuntu	118M	0.0
148391	lazygit	lazygit	14	ubuntu	32M	0.2
4179	snapedesktop-i	/snap/snapedesktop-i	13	ubuntu	91M	0.0
1804	snaped	/usr/lib/snaped/snaped	12	root	42M	0.0
5247	gjs	/usr/share/gnome-	11	ubuntu	69M	0.0
3656	evolution-calen	/usr/libexec/evolutio	10	ubuntu	23M	0.0
5955	snaped	/usr/bin/snaped userd	9	ubuntu	25M	0.0
125572	eog	/usr/bin/eog /home/ub	8	ubuntu	67M	0.0

Top CPU users are firefox and gnome-shell (core GUI for linux environment), which makes sense as they are the most complex applications running currently.

To become a top user we would need to open more firefox tabs (consume more RAM) or, switch screens quickly to put load on the GUI interface.

2.



The CPU appears to never be idle as there are always background processes running even just to keep the screen on, or to be listening for keyboard inputs, or mouse clicks, or any other possible input/ change like wifi connection/ USB drive status. From the program list we can see that even when nothing is happening there are hundreds of programmes still running.

Resource Usage – Compute Power, Energy and Thermals

```
ubuntu@ubuntu:~/CWM-FDI/assignment3$ sudo turbostat -q -S --show PkgWatt -i 1
PkgWatt
2.53
2.70
2.59
3.09
6.28
2.81
2.78
3.00
4.75
2.88
2.76
6.24
4.36
4.88
```

```
ubuntu@ubuntu:~/CWM-FDI/assignment3$ sudo turbostat -q -S --Joules --show Pkg_J -i 1
Pkg_J
2.48
2.67
3.06
2.79
2.69
2.63
2.71
2.71
2.50
2.54
2.52
2.56
2.77
5.00
2.82
```

3. The baseline very much fluctuates over time, as the OS is continuously reallocating resources to whatever it feels requires them at that instant. Most interestingly, we see spikes whenever I switched windows the firefox/ libreoffice writer window (resource intensive), away from the terminals window (not resource intensive). This is likely the OS reallocating resources to the windows that need it to improve efficiency, ie. so that we are not running firefox at full capacity when we cannot even see it.

4.

a)

```
ubuntu@ubuntu:~/CWM-FDI/assignment3$ sudo turbostat -q -S --show Busy%,Bzy_MHz,PkgWatt,CorWatt,PkgTmp,CoreTmp -i 1
Busy%   Bzy_MHz  CoreTmp  PkgTmp  PkgWatt  CorWatt
1.55    3591    34       35      2.87     1.12
1.18    3581    35       35      2.66     0.92
0.86    3573    35       35      2.58     0.84
1.23    3592    34       35      2.67     0.93
0.69    3576    35       36      2.28     0.55
1.27    3588    35       35      2.55     0.81
0.76    3597    34       35      2.36     0.62
1.09    3588    34       35      2.64     0.90
1.85    3588    35       35      2.77     1.03
1.21    3588    34       35      2.69     0.95
1.10    3595    34       35      2.63     0.89
```

b) the business metric compares pretty well, as we can see the idle % sitting at around 1%, matching the %CPU measurement in the diagram using btop from Q2 (sat at around 1-2%).

this tool shows more clearly how idleness varies over time, which the other tools htop/btop/ps aux make it harder to see.

5.

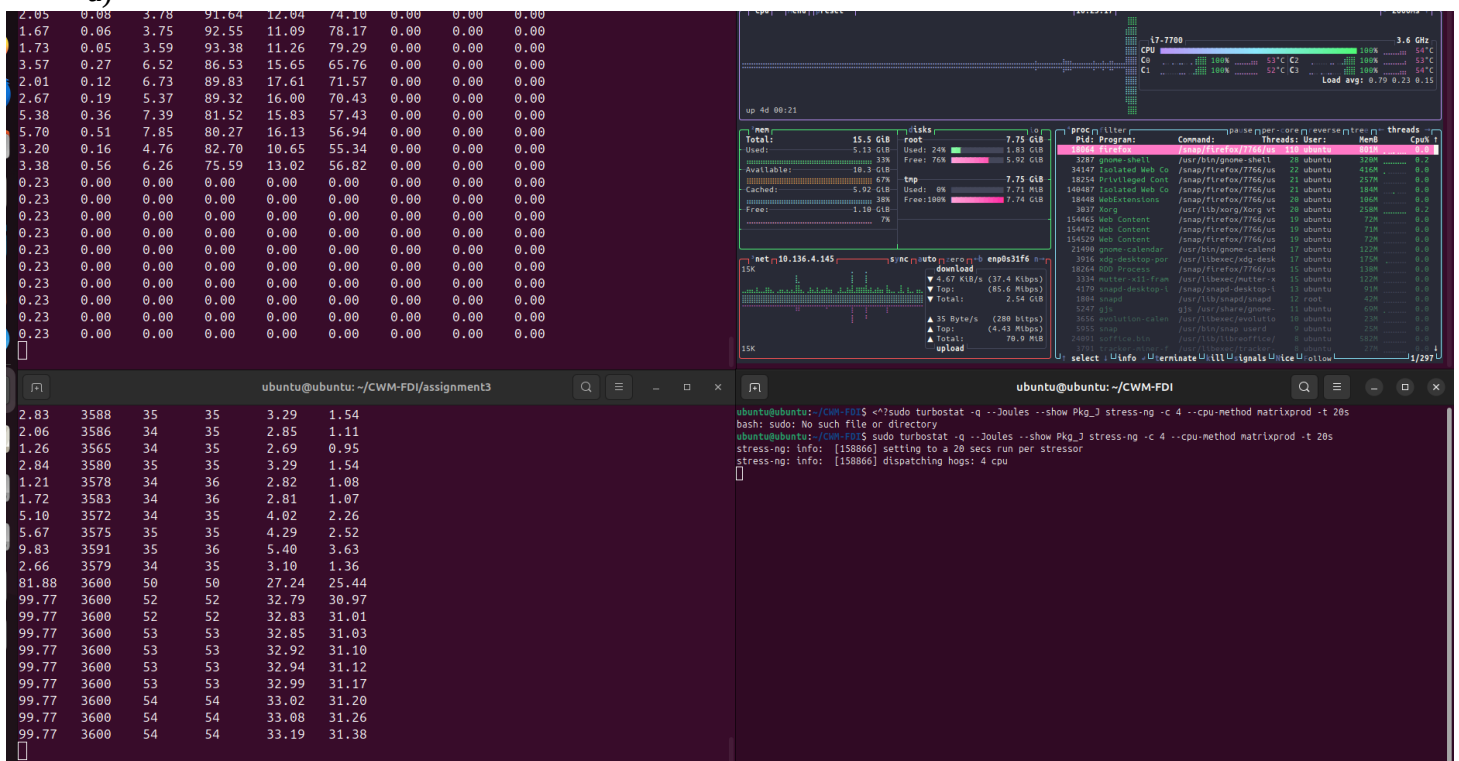
```
ubuntu@ubuntu:~/CWM-FDI/assignment3$ sudo turbostat -q -S --show Pkg%pc2,Pkg%pc3,Pkg%pc6,Pkg%pc7,Pkg%pc8,CPU%c1,CPU%c3,CPU%c6,CPU%c7 -i 1
```

CPU%c1	CPU%c3	CPU%c6	CPU%c7	Pkg%pc2	Pkg%pc3	Pkg%pc6	Pkg%pc7	Pkg%pc8
1.73	0.09	4.64	92.38	12.63	79.07	0.00	0.00	0.00
1.86	0.08	3.33	93.54	10.59	80.23	0.00	0.00	0.00
1.34	0.07	3.30	94.28	10.41	82.47	0.00	0.00	0.00
1.63	0.04	4.85	92.12	10.85	78.42	0.00	0.00	0.00
1.51	0.05	4.07	93.30	10.87	81.09	0.00	0.00	0.00
1.31	0.01	2.41	95.54	10.98	82.02	0.00	0.00	0.00
1.32	0.05	2.95	94.62	10.60	82.02	0.00	0.00	0.00
1.20	0.03	2.80	95.22	10.36	82.90	0.00	0.00	0.00
1.38	0.04	3.78	89.23	9.00	71.60	0.00	0.00	0.00
1.84	0.04	3.49	91.11	9.68	73.93	0.00	0.00	0.00
1.64	0.15	3.53	93.63	11.11	80.90	0.00	0.00	0.00

The Impact of Invoking New Commands

6.

a)

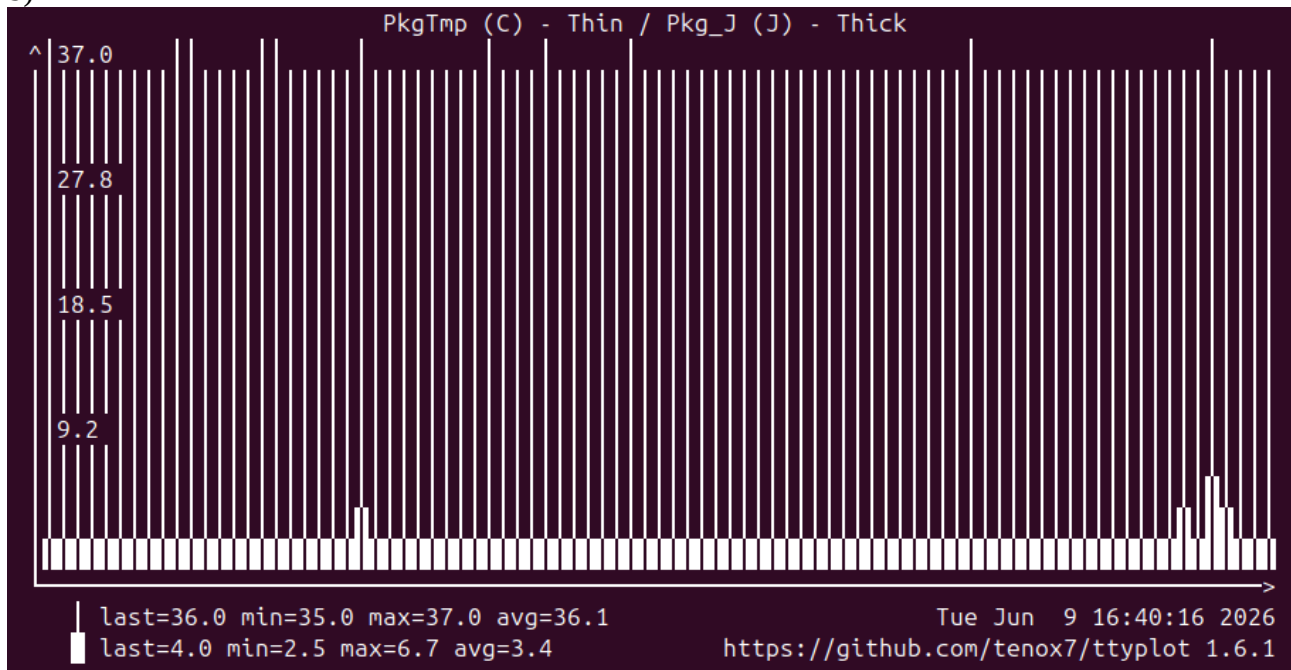


we can see the CPU usage % spiking massively to almost 100%, and we can see the power usage jumping up to 31W nearly

```
proc filter | pause | per-core | reverse | tree | cpu direct
```

Pid:	Program:	Command:	Threads:	User:	MemB	Cpu%
159048	stress-ng-cpu	stress-ng-cpu [run]	1	root	3.3M	24.8
159049	stress-ng-cpu	stress-ng-cpu [run]	1	root	3.3M	24.8
159050	stress-ng-cpu	stress-ng-cpu [run]	1	root	3.3M	24.5
159047	stress-ng-cpu	stress-ng-cpu [run]	1	root	3.3M	24.4

b)



c) Idle joules sits at around 5J, similar to what the other tools showed, and much lower than what it was during stress testing. This under-utilization means we are wasting energy while sitting idle and it is not being used for anything - highly inefficient.

7.

```
ubuntu@ubuntu:~/CWM-FDI/assignment3$ sudo turbostat -q --Joules --show Pkg_J python3 ../assignment1/matmul_slow.p
500
n=500 reps=2 checksum=477458.000000
avg cell time: 3.436614663444925e-05
avg calc time: 8.624890760489507
17.583389 sec
Pkg_J
190.08
190.08

ubuntu@ubuntu:~/CWM-FDI/assignment3$ sudo turbostat -q --Joules --show Pkg_J python3 ../assignment1/matmul_fast.p
500
n=500 reps=2 checksum=477458.000000
matmul_fast3 avg time = 5.417808550497284
10.963745 sec
Pkg_J
120.31
120.31
```

The fast version had an energy consumption of 120J, vs the 190J of the slow version. Initially my intuition was that we would trade program speed for efficiency, and the faster program would be less efficient, but now I see that speed and efficiency are linked, and the program is essentially faster precisely because it is more efficient.

8. Yes, there were packages and cores in deeper sleep states, fairly often.

Some possible downsides of deeper sleep is slower response times when those cores are actually then needed – deeper sleep may take longer to come out of. Applications where response times

matter would be most adversely affected by sleep states, as the extra latency introduced by waking up from the sleep state really matters in those scenarios. Eg. games/ use facing applications etc.

9.

density vs cooling trade off where a higher utilization improves profit, but requires expensive cooling upgrades, and the thermals become the limiting factor as the profit curve flattens as increased utilization requires increased cooling and the cost of one cancels out the gains of the other

thermals currently limit scaling as thermal limits force discrete expansion steps like new buildings/ sits so there is a longer lead time than desired and prediction of future requirements becomes the main issue.

We can also tackle density and thermals by spreading out data centers in different locations, keeping per data center utilization lower and avoiding thermal issues, but this in turn increases complexity and operational friction.

Network Activity

10.

- a) at 1Gb/s we would of course assume that it would take ~ 1 s to transmit a 1Gb file at full data rate
- b) in 1s, assuming the power draw of a 1Gb/s ethernet port is 0.3W per 1W of active power, we would assume 0.3J/ Gb
- c) if the data has to pass through multiple machines, then we will be passing through multiple ethernet ports each with their own power draws which would sum along the chain leading to a much lower energy efficiency of data transfer

would need to take into account any other possible energy use cases ie. loss in signal generation, usage in processing when moving between servers, etc...

11.

a)

b)

```
sg_90k_part1.json?down 100%[=====] 878.89M 111MB/s in 7.9s
2026-06-10 11:22:09 (111 MB/s) - 'sg_90k_part1.json?download=true' saved [921586083/921586083]
8.131300 sec
Pkg_J
60.18
60.18
```

c)

theoretical results were 0.3J/ Gb and this was 60.18J/ 878.89 Mb, ie. 64J /Gb, which is over 200x worse than theory. This suggests that the dominant form of energy usage is not the port itself but the processing of the data once it arrives at the computer or any other form of energy use in the chain.

12.

For 480p we average at about 6W of CPU power usage

For 1080p we average at about 8W of CPU power usage

For 2160p, we average at about 20W of CPU power usage

We can clearly see a massive jump in power between 1080p and 2160p, compared to a relatively small power jump between 480p and 1080. This shows us that video streaming gets drastically less energy efficient as we increase the streaming quality.

Thought Experiments – Carbon Footprint

13.

Current carbon intensity in England is 122gCO₂ / kWh ie. 33.89gCO₂ / MJ

Carbon Intensity in Spain is 81gCO₂ / kWh at Midday and 100gCO₂ / kWh at midnight

The programmatic improvement gave us nearly 40% energy savings, which would be a massive improvement at scale. From these numbers above, we see that the carbon footprint add midday is lower than at midnight, so running programs during the day would have a lower footprint than at night, and running them in Spain rather than in England, or more generally in countries that rely more heavily on green energy could further reduce the footprint of these programs.

14.

0.34 Wh/query * 2.5B queries/ day * 100gCO₂/kWh gives us:

85 metric tonnes of CO₂/day

2550 metric tonnes of CO₂/month

31000 metric tonnes of CO₂/year

This is around 150x the cost of training GPT4 (100-200 tonnes of CO₂)

This is equivalent to around 7000 tesla's driven 50km/day continuously

This is around 2600-3000 times a typical household's annual emissions

0.2% of San Francisco's annual footprint

236000 gaming desktops (150W) running 24/7

power/util chart -correlate with frequency, DVFS